

第二十五章 TFT-LCD（MCU 屏）实验

前面我们介绍了 OLED 模块及其显示，但是该模块只能显示单色/双色，不能显示彩色，而且尺寸也较小。本章我们将介绍正点原子的 TFT-LCD 模块（MCU 屏），该模块采用 TFT-LCD 面板，可以显示 16 位色的真彩图片。在本章中，我们将使用开发板底板上的 TFTLCD 接口（仅支持 MCU 屏，本章仅介绍 MCU 屏的使用），来点亮 TFT-LCD，并实现 ASCII 字符和彩色的显示等功能，并在串口打印 LCD 控制器 ID，同时在 LCD 上面显示。

本章分为如下几个小节：

25.1 TFTLCD 和 FSMC 简介

25.2 硬件设计

25.3 程序设计

25.4 下载验证

25.1 TFT-LCD 简介

本章我们将通过 STM32F103 的 FSMC 外设来控制 TFT-LCD 的显示，这样我们就可以用 STM32 输出一些信息到显示屏上了。

25.1.1 TFT-LCD 简介

液晶显示器，即 Liquid Crystal Display，利用了液晶导电后透光性可变的特性，配合显示器光源、彩色滤光片和电压控制等工艺，最终可以在液晶阵列上显示彩色的图像。目前液晶显示技术以 TN、STN、TFT 三种技术为主，TFT-LCD 即采用了 TFT（Thin Film Transistor）技术的液晶显示器，也叫薄膜晶体管液晶显示器。

TFT-LCD 与无源 TN-LCD、STN-LCD 的简单矩阵不同的是，它在液晶显示屏的每一个像素上都设置有一个薄膜晶体管（TFT），可有效地克服非选通时的串扰，使显示液晶屏的静态特性与扫描线数无关，因此大大提高了图像质量。TFT 式显示器具有很多优点：高响应度，高亮度，高对比度等等。TFT 式屏幕的显示效果非常出色，广泛应用于手机屏幕、笔记本电脑和台式机显示器上。

由于液晶本身不会发光，加上液晶本身的特性等原因，使得液晶屏的成像角受限，我们从屏幕的一侧可能无法看清液晶的显示内容。液晶显示器的成像角的大小也是评估一个液晶显示器优劣的指标，目前，规格较好的液晶显示器成像角一般在 120°~160°之间。

正点原子 TFT-LCD 模块(MCU 屏)有如下特点：

- 1，2.8' /3.5' /4.3' /7' 等 4 种大小的屏幕可选。
- 2，320×240 的分辨率（3.5' 分辨率为：320*480，4.3' 和 7' 分辨率为：800*480）。
- 3，16 位真彩显示。
- 4，自带触摸屏，可以用来作为控制输入。

本章，我们以正点原子 2.8 寸（此处的寸是代表英寸，下同）的 TFT-LCD 模块为例介绍，（其他尺寸的 LCD 可参考具体的 LCD 型号的资料，也比较类似），该模块支持 65K 色显示，显示分辨率为 320×240，接口为 16 位的 8080 并口，自带触摸功能。

该模块的外观如图 25.1.1.1 所示：



图 25.1.1.1 正点原子 2.8 寸 TFTLCD 外观图

模块原理图如图 25.1.1.2 所示：

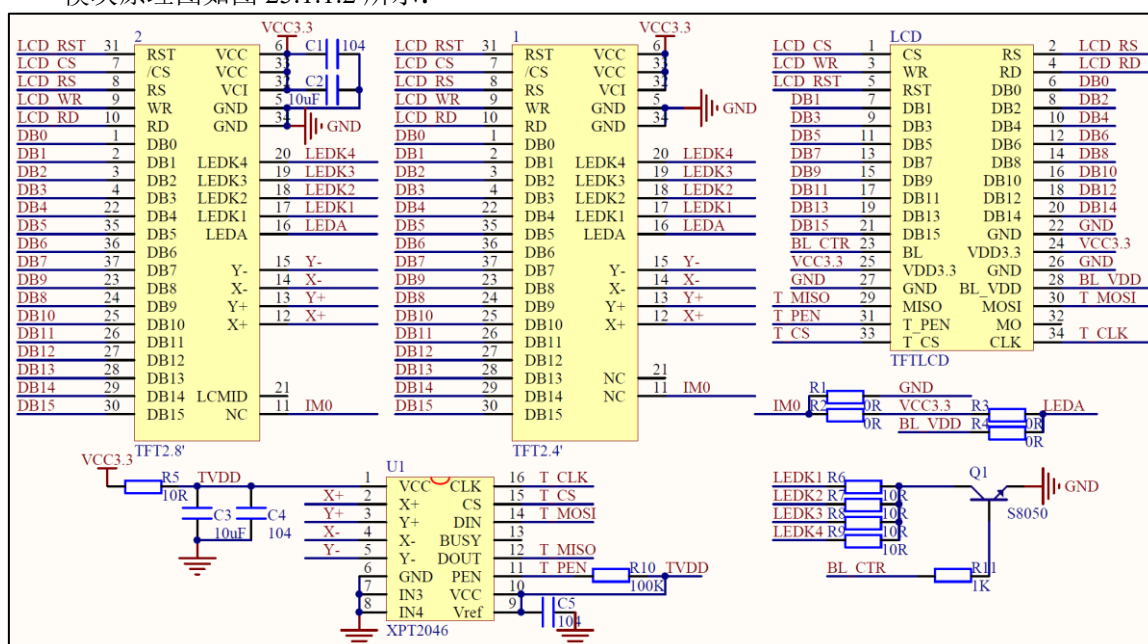


图 25.1.1.2 正点原子 TFTLCD 模块原理图

TFTLCD 模块采用 2*17 的 2.54 公排针与外部连接，即图中 TFT_LCD 部分。从图 25.1.1.2 可以看出，正点原子 TFTLCD 模块采用 16 位的并方式与外部连接。图 25.1.1.2 还列出了触摸控制的接口，但触摸控制是在显示的基础上叠加的一个控制功能，不配置也不会对显示造成影响，我们放到以后的章节再介绍触摸的用法。该模块与显示功能的相关的信号线如表 25.1.1.1：

名称	功能
CS	TFT-LCD 片选信号。
WR	向 TFT-LCD 写入数据。
RD	从 TFT-LCD 读取数据。
DB[15:0]	16 位双向数据线。
RST	硬复位 TFT-LCD。
RS	命令/数据标志（0，读写命令；1，读写数据）。
BL	背光控制。

表 25.1.1.1 TFT-LCD 接口信号线

上述的接口线实际是对应到液晶显示控制器上的，这个芯片位于液晶屏的下方，所以我们从外观图上看不到。控制 LCD 显示的过程，就是按其显示驱动芯片的时序，把色彩和位置信息正确地写入对应的寄存器。

25.1.2 液晶显示控制器

正点原子提供 2.8/3.5/4.3/7 寸等 4 种不同尺寸和分辨率的 TFTLCD 模块，其驱动芯片为：ILI9341/ST7789/NT35310/NT35510/SSD1963 等(具体的型号,大家可以通过下载本章实验代码,通过串口或者 LCD 显示查看),这里我们仅以 ILI9341 控制器为例进行介绍,其他的控制基本都类似,我们就不详细阐述了。

ILI9341 液晶控制器自带显存,可配置支持 8/9/16/18 位的总线中的一种,可以通过 3/4 线串行协议或 8080 并口驱动。正点原子的 TFT-LCD 模块上的电路配置为 8080 并口方式,其显存总大小为 172800 (240*320*18/8),即 18 位模式 (26 万色) 下的显存量。在 16 位模式下,ILI9341 采用 RGB565 格式存储颜色数据,此时 ILI9341 的 18 位显存与 MCU 的 16 位数据线以及 RGB565 的对应关系如图 25.1.2.1 所示:

9341 显存	B17	B16	B15	B14	B13	B12	B11	B10	B9	B8	B7	B6	B5	B4	B3	B2	B1	B0
RGB565 GRAM	R[4]	R[3]	R[2]	R[1]	R[0]	NC	G[5]	G[4]	G[3]	G[2]	G[1]	G[0]	B[4]	B[3]	B[2]	B[1]	B[0]	NC
9341 数据线	DB15	DB14	DB13	DB12	DB11		DB10	DB9	DB8	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
MCU 数据线	D15	D14	D13	D12	D11		D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	

图 25.1.2.1 16 位数据与显存对应关系图

从图中可以看出,ILI9341 在 16 位模式下面,18 位显存的 B0 和 B12 并没有用到,对外的数据线使用 DB0-DB15 连接 MCU 的 D0-D15 实现 16 位颜色的传输 (使用 8080 MCU 16bit I 型接口,详见 9341 数据手册 7.1.1 节)。

这样 MCU 的 16 位数据,最低 5 位代表蓝色,中间 6 位为绿色,最高 5 位为红色。数值越大,表示该颜色越深。另外,特别注意 ILI9341 所有的指令都是 8 位的 (高 8 位无效),且参数除了读写 GRAM 的时候是 16 位,其他操作参数,都是 8 位的。

知道了屏幕的显色信息后,我们如何驱动它呢? OLED 的章节我们已经描述过 8080 方式操作的时序,我们通过《ILI9341_DS.pdf》来加深一下在 8080 并口方式下如何操作这个芯片。

以写周期为例,8080 方式下的操作时序如图 25.1.2.2 所示。

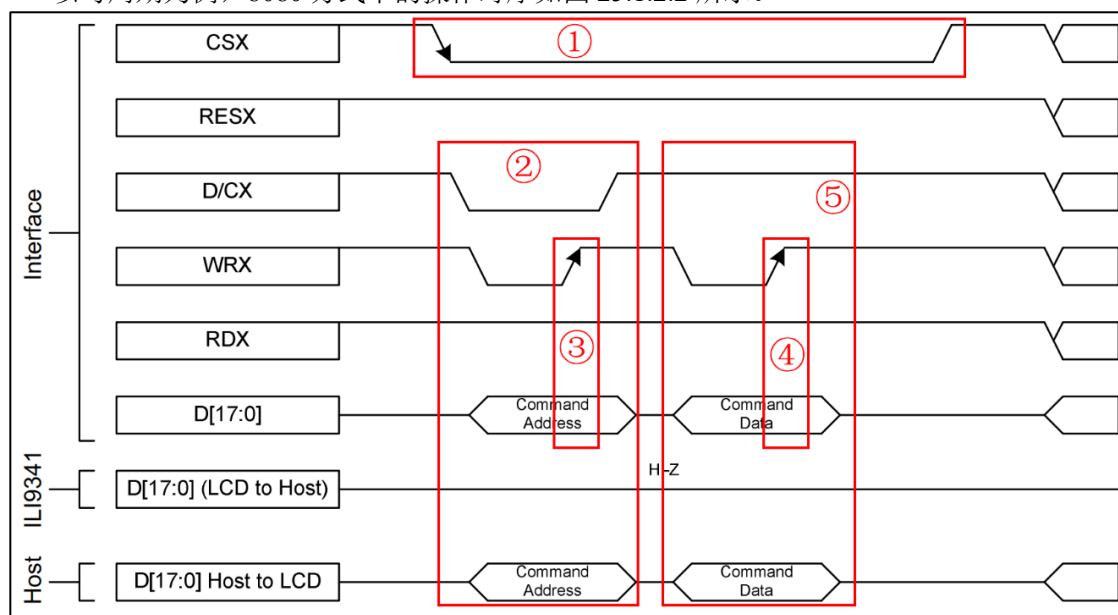


图 25.1.2.2 8080 方式下对液晶控制器的写操作

上图中的各个控制线与我们在表 25.1.1.1 提到的命名有些许差异,因为我们在原理图时往往为了方便自己记忆会对命名进行微调,为了方便读者对照,我们把图 25.1.2.2 中列出的引脚与我们的 TFTLCD 模块的对应关系再列出,如表 25.1.2.1 所示。

TFTLCD 名称	ILI9341	功能
CS	CSX	TFT-LCD 片选信号。
WR	WRX	向 TFT-LCD 写入数据。
RD	RDX	从 TFT-LCD 读取数据。
D[15:0]	D[15:0]	16 位双向数据线。

RS	D/CX	命令/数据标志（0，读写命令；1，读写数据）。
----	------	-------------------------

表 25.1.2.1 TFT-LCD 引脚与液晶控制器的对应关系

这下我们再分析一下图 25.1.2.2 所示的写操作的时序，控制液晶的主机，在整个写周期内需要控制片选 CSX 拉低（标注为①），之后对其它的控制线的电平才有效。在标号②表示的这个写命令周期中，D/CX 被位低（参考 ILI9341 的引脚定义），同时把命令码通过数据线 D[17:0]（我们实际只用了 16 个引脚）按位编码。注意到③处，需要数据线在入电平拉高后再操持一段时间以便数据被正确采样。

图 25.1.2.2 中⑤表示写数据操作，与前面描述的写命令操作只有 D/CX 的操作不同，读者们可以尝试自己分析一下。更多的关于 ILI9341 的读写操作时序则参考《ILI9341_DS.pdf》。

通过前述的时序分析，我们知道了对于 ILI9341 来说，控制命令有命令码、数据码之分，接下来，我们介绍一下 ILI9341 的几个重要命令。因为 ILI9341 的命令很多，我们这里就不全部介绍了，有兴趣的大家可以找到 ILI9341 的 datasheet 看看。里面对这些命令有详细的介绍。我们将介绍：0xD3，0x36，0x2A，0x2B，0x2C，0x2E 等 6 条指令。

指令 0xD3，是读 ID4 指令，用于读取 LCD 控制器的 ID，该指令如表 25.1.2.2 所示：

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	1	1	0	1	0	0	1	1	D3H
参数 1	1	↑	1	XX	X	X	X	X	X	X	X	X	X
参数 2	1	↑	1	XX	0	0	0	0	0	0	0	0	00H
参数 3	1	↑	1	XX	1	0	0	1	0	0	1	1	93H
参数 4	1	↑	1	XX	0	1	0	0	0	0	0	1	41H

表 25.1.2.2 0xD3 指令描述

从上表可以看出，0xD3 指令后面跟了 4 个参数，最后 2 个参数，读出来是 0x93 和 0x41，刚好是我们控制器 ILI9341 的数字部分，从而，通过该指令，即可判别所用的 LCD 驱动器是什么型号，这样，我们的代码，就可以根据控制器的型号去执行对应驱动 IC 的初始化代码，从而兼容不同驱动 IC 的屏，使得一个代码支持多款 LCD。

接下来看指令：0x36，这是存储访问控制指令，可以控制 ILI9341 存储器的读写方向，简单的说，就是在连续写 GRAM 的时候，可以控制 GRAM 指针的增长方向，从而控制显示方式（读 GRAM 也是一样）。该指令如表 25.1.2.3 所示：

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	1	0	1	1	0	36H
参数	1	1	↑	XX	MY	MX	MV	ML	BGR	MH	0	0	0

表 25.1.2.3 0x36 指令描述

从上表可以看出，0x36 指令后面，紧跟一个参数，这里主要关注：MY、MX、MV 这三个位，通过这三个位的设置，我们可以控制整个 ILI9341 的全部扫描方向，如表 25.1.2.4 所示：

控制位			效果
MY	MX	MV	
0	0	0	从左到右,从上到下
1	0	0	从左到右,从下到上
0	1	0	从右到左,从上到下
1	1	0	从右到左,从下到上
0	0	1	从上到下,从左到右
0	1	1	从上到下,从右到左
1	0	1	从下到上,从左到右
1	1	1	从下到上,从右到左

表 25.1.2.4 MY、MX、MV 设置与 LCD 扫描方向关系表

这样，我们在利用 ILI9341 显示内容的时候，就有很大的灵活性了，比如显示 BMP 图片，BMP 解码数据，就是从图片的左下角开始，慢慢显示到右上角，如果设置 LCD 扫描方向为从

左到右，从下到上，那么我们只需要设置一次坐标，然后就不停的往 LCD 填充颜色数据即可，这样可以大大提高显示速度。

实验中，我们默认使用从左到右，从上到下的扫描方式。

接下来看指令：0x2A，这是列地址设置指令，在从左到右，从上到下的扫描方式（默认）下面，该指令用于设置横坐标（x 坐标），该指令如表 25.1.2.5 所示：

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	0	1	0	2AH
参数 1	1	1	↑	XX	SC15	SC14	SC13	SC12	SC11	SC10	SC9	SC8	SC
参数 2	1	1	↑	XX	SC7	SC6	SC5	SC4	SC3	SC2	SC1	SC0	
参数 3	1	1	↑	XX	EC15	EC14	EC13	EC12	EC11	EC10	EC9	EC8	EC
参数 4	1	1	↑	XX	EC7	EC6	EC5	EC4	EC3	EC2	EC1	EC0	

表 25.1.2.5 0x2A 指令描述

在默认扫描方式时，该指令用于设置 x 坐标，该指令带有 4 个参数，实际上是 2 个坐标值：SC 和 EC，即列地址的起始值和结束值，SC 必须小于等于 EC，且 $0 \leq SC/EC \leq 239$ 。一般在设置 x 坐标的时候，我们只需要带 2 个参数即可，也就是设置 SC 即可，因为如果 EC 没有变化，我们只需要设置一次即可（在初始化 ILI9341 的时候设置），从而提高速度。

与 0x2A 指令类似，指令：0x2B，是页地址设置指令，在从左到右，从上到下的扫描方式（默认）下面，该指令用于设置纵坐标（y 坐标）。该指令如表 25.1.2.6 所示：

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	0	1	1	2BH
参数 1	1	1	↑	XX	SP15	SP14	SP13	SP12	SP11	SP10	SP9	SP8	SP
参数 2	1	1	↑	XX	SP7	SP6	SP5	SP4	SP3	SP2	SP1	SP0	
参数 3	1	1	↑	XX	EP15	EP14	EP13	EP12	EP11	EP10	EP9	EP8	EP
参数 4	1	1	↑	XX	EP7	EP6	EP5	EP4	EP3	EP2	EP1	EP0	

表 25.1.2.6 0x2B 指令描述

在默认扫描方式时，该指令用于设置 y 坐标，该指令带有 4 个参数，实际上是 2 个坐标值：SP 和 EP，即页地址的起始值和结束值，SP 必须小于等于 EP，且 $0 \leq SP/EP \leq 319$ 。一般在设置 y 坐标的时候，我们只需要带 2 个参数即可，也就是设置 SP 即可，因为如果 EP 没有变化，我们只需要设置一次即可（在初始化 ILI9341 的时候设置），从而提高速度。

接下来看指令：0x2C，该指令是写 GRAM 指令，在发送该指令之后，我们便可以往 LCD 的 GRAM 里面写入颜色数据了，该指令支持连续写，指令描述如表 25.1.2.7 所示。

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	1	0	0	2CH
参数 1	1	1	↑	D1[15: 0]									XX
.....	1	1	↑	D2[15: 0]									XX
参数 n	1	1	↑	Dn[15: 0]									XX

表 25.1.2.7 0x2C 指令描述

由表 25.1.2.7 可知，在收到指令 0x2C 之后，数据有效位宽变为 16 位，我们可以连续写入 LCD GRAM 值，而 GRAM 的地址将根据 MY/MX/MV 设置的扫描方向进行自增。例如：假设设置的是从左到右，从上到下的扫描方式，那么设置好起始坐标（通过 SC，SP 设置）后，每写入一个颜色值，GRAM 地址将会自动自增 1（SC++），如果碰到 EC，则回到 SC，同时 SP++，一直到坐标：EC，EP 结束，期间无需再次设置的坐标，从而大大提高写入速度。

最后，来看看指令：0x2E，该指令是读 GRAM 指令，用于读取 ILI9341 的显存（GRAM），该指令在 ILI9341 的数据手册上面的描述是有误的，真实的输出情况如表 25.1.2.8 所示：

顺序	控制			各位描述 D[15:0]									HEX
	RS	RD	WR	15~11	10~8	7	6	5	4	3	2	1	0

指令	0	1	↑	XX		0	0	1	0	1	1	1	0	2EH
参数 1	1	↑	1	XX										dummy
参数 2	1	↑	1	R1[4:0]	XX	G1[5:0]						XX	R1G1	
参数 3	1	↑	1	B1[4:0]	XX	R2[4:0]					XX	B1R2		
参数 4	1	↑	1	G2[5:0]		XX	B2[4:0]					XX	G2B2	
参数 5	1	↑	1	R3[4:0]	XX	G3[5:0]						XX	R3G3	
参数 N	1	↑	1	按以上规律输出										

表 25.1.2.8 0X2E 指令描述

该指令用于读取 GRAM，如表 25.1.2.7 所示，ILI9341 在收到该指令后，第一次输出的是 dummy 数据，也就是无效的数据，第二次开始，读取到的才是有效的 GRAM 数据（从坐标：SC，SP 开始），输出规律为：每个颜色分量占 8 个位，一次输出 2 个颜色分量。比如：第一次输出是 R1G1，随后的规律为：B1R2→G2B2→R3G3→B3R4→G4B4→R5G5...以此类推。如果我们只需要读取一个点的颜色值，那么只需要接收到参数 3 即可，如果要连续读取（利用 GRAM 地址自增，方法同上），那么就按照上述规律去接收颜色数据。

以上，就是操作 ILI9341 常用的几个指令，通过这几个指令，我们便可以很好的控制 ILI9341 显示我们所要显示的内容了。

25.1.3 FSMC 简介

ILI9341 的 8080 通讯接口时序可以由 STM32 使用 GPIO 接口进行模拟，但这样效率太低，STM32 提供了一种更高效的控制方法——使用 FSMC 接口实现 8080 时序，但 FSMC 是 STM32 片上外设的一种，并非所有的 STM32 都拥有这种硬件接口，使用何种方式驱动需要在芯片选型时就确定好。我们的开发板支持 FSMC 接口，下面我们来了解一下这个接口的功能。

FSMC，即灵活的静态存储控制器，能够与同步或异步存储器和 16 位 PC 存储器卡连接，FSMC 接口可以通过地址信号，快速找到存储器对应存储块上的数据。STM32F1 的 FSMC 接口支持包括 SRAM、NAND FLASH、NOR FLASH 和 PSRAM 等存储器。F1 系列的大容量型号，且引脚数目在 100 脚及以上的 STM32F103 芯片都带有 FSMC 接口，正点原子战舰 STM32F103 的主芯片为 STM32F103ZET6，是带有 FSMC 接口的。

FSMC 接口的结构如图 25.1.3.1 所示：

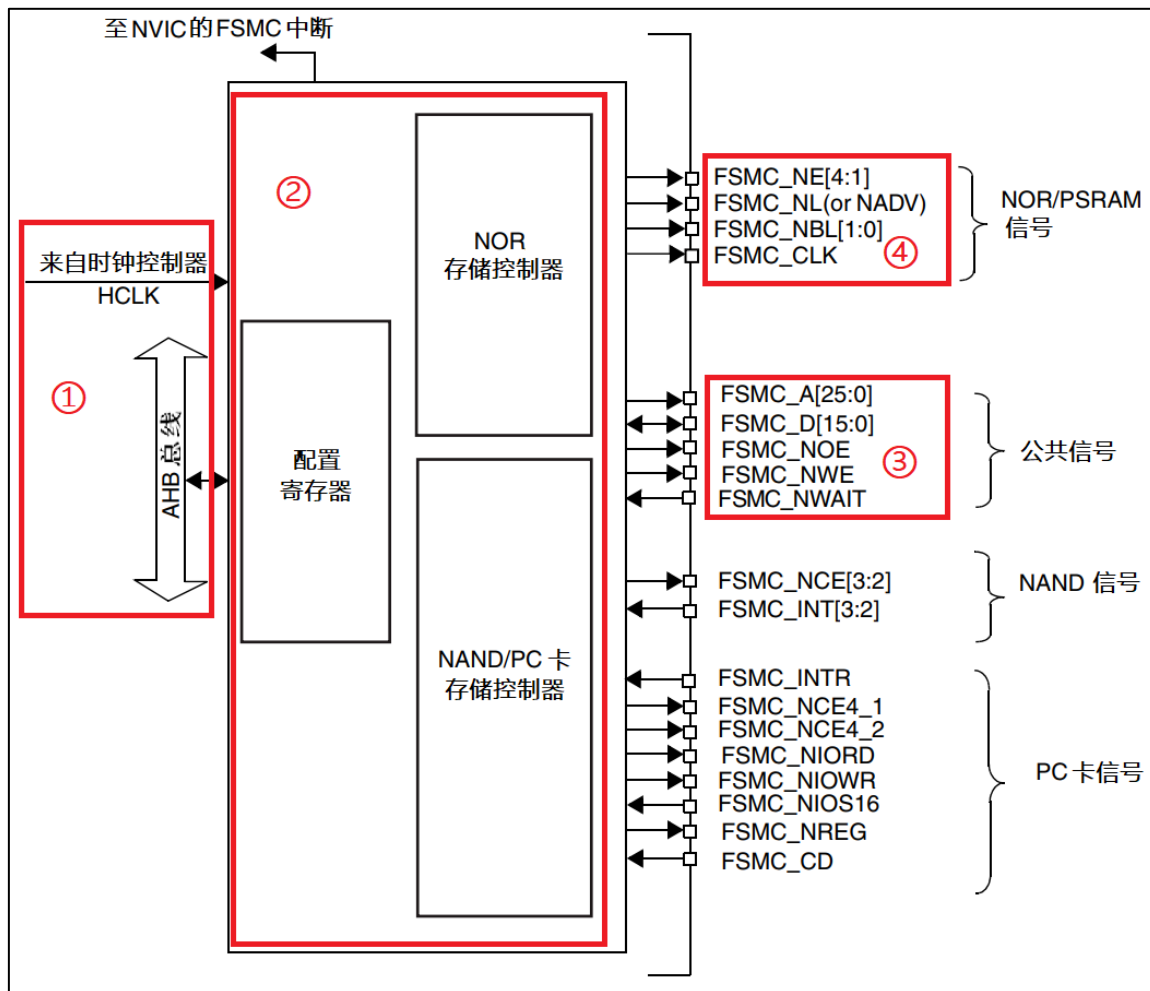


图 25.1.3.1 FSMC 框图

从图 25.1.3.1 我们可以看出，STM32 的 FSMC 可以驱动 NOR/PSRAM、NAND、PC 卡这 3 类设备，他们具有不同的 CS 以区分不同的设备。本部分我们要用到的是 NOR/PSRAM 的功能。①为 FSMC 的总线和时钟源，②为 STM32 内部的 FSMC 控制单元，③是连接硬件的引脚，这里的“公共信号”表示不论我们驱动前面提到的 3 种设备中的哪种，这些 IO 是共享的，所以如果需要用到多种功能的情况，程序上还要考虑分时复用。④是 NOR/PSRAM 会使用到的信号控制线，③和④这些信号比较重要，它们的功能如表 25.1.3.1：

FSMC 信号名称	信号方向	功能
FSMC_NE[x]	输出	STM32F1 有四个片选引脚，x = 1...4，每个对应不同的内存块
FSMC_CLK	输出	时钟(同步突发模式使用)
FSMC_A[25:0]	输出	地址总线
FSMC_D[15:0]	输入/输出	双向数据总线
FSMC_NOE	输出	输出使能
FSMC_NWE	输出	写使能
FSMC_NWAIT	输入	NOR 闪存要求 FSMC 等待的信号
FSMC_NADV	输出	地址、数据线复用时作锁存信号

表 25.1.3.1 FSMC 信号线的功能

在数电的课程中有介绍过存储器的知识，它是可以存储数据的器件。复杂的存储器为了存储更多的数据，常常通过地址线来管理数据存储的位置，这样只要先找到需要读写数据的位置，然后对数据进行读写的操作。由于存储器的这种数据和地址对应关系，采用 FSMC 这种专门硬件接口就能加快对存储器的数据访问。

STM32F1 的 FSMC 将外部存储器划分为固定大小为 256M 字节的四个存储块，FSMC 的外部设备地址映像如图 25.1.3.2 所示：

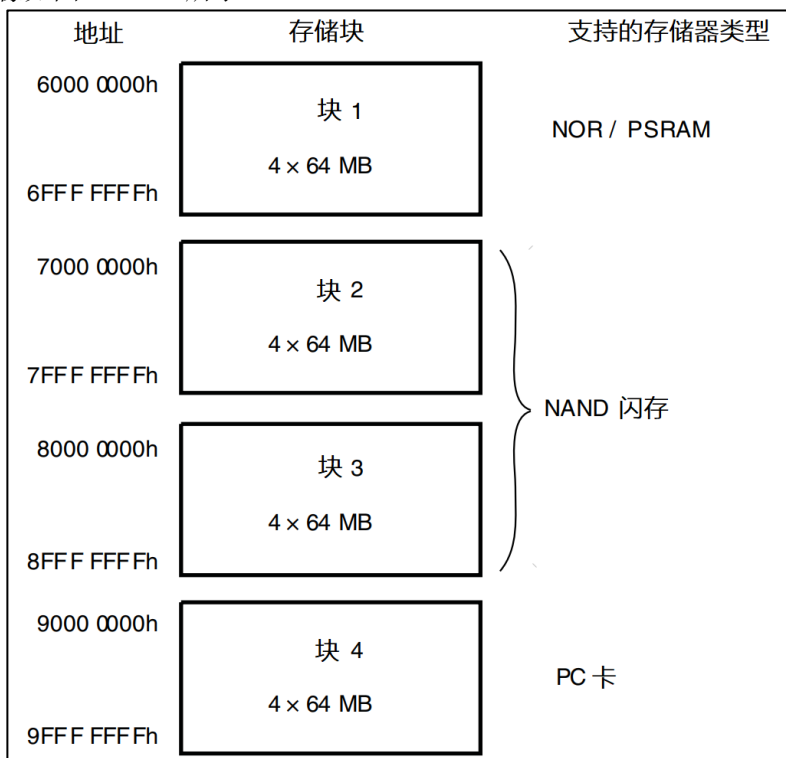


图25.1.3.2 FSMC存储块地址映像

从上图可以看出，FSMC 总共管理 1GB 空间，拥有 4 个存储块（Bank），FSMC 各 Bank 配置寄存器如表 25.1.3.2：

内部控制器	存储块	管理的地址范围	支持的设备类型	配置寄存器
NOR FLASH 控制器	Bank1	0X6000,0000~0X6FFF,FFFF	SRAM/ROM	FSMC_BCR1/2/3/4
			NOR FLASH	FSMC_BTR1/2/2/3
			PSRAM	FSMC_BWTR1/2/3/4
NAND FLASH PC CARD 控制器	Bank2	0X7000,0000~0X7FFF,FFFF	NAND FLASH	FSMC_PCR2/3/4
	Bank3	0X8000,0000~0X8FFF,FFFF		FSMC_SR2/3/4
	Bank4	0X9000,0000~0X9FFF,FFFF	PC Card	FSMC_PMEM2/3/4 FSMC_PATT2/3/4 FSMC_PIO4

表25.1.3.2 FSMC各Bank配置寄存器表

本章，我们用到的是块 1，所以在本章我们仅讨论块 1 的相关配置，其他块的配置，请参考《STM32F10xxx 参考手册_V10（中文版）.pdf》第 19 章（324 页）的相关介绍。

STM32F1 的 FSMC 存储块 1（Bank1）被分为 4 个区，每个区管理 64M 字节空间，每个区都有独立的寄存器对所连接的存储器进行配置。Bank1 的 256M 字节空间可以通过 28 根地址线（HADDR[27:0]）寻址后访问。这里 HADDR 是内部 AHB 地址总线，其中 HADDR[25:0]来自外部存储器地址 FSMC_A[25:0]，而 HADDR[26:27]对 4 个区进行寻址。如表 25.1.3.3 所示：

Bank1 所选区	片选信号	地址范围	HADDR	
			[27:26]	[25:0]
第 1 区	FSMC_NE1	0X6000,0000~63FF,FFFF	00	FSMC_A[25:0]
第 2 区	FSMC_NE2	0X6400,0000~67FF,FFFF	01	
第 3 区	FSMC_NE3	0X6800,0000~6BFF,FFFF	10	
第 4 区	FSMC_NE4	0X6C00,0000~6FFF,FFFF	11	

表 25.1.3.3 Bank1 存储区选择表

表 25.1.3.3 中，我们要特别注意 HADDR[25:0]的对应关系：

当 Bank1 接的是 16 位宽度存储器的时候：HADDR[25:1]→FSMC_A[24:0]。

当 Bank1 接的是 8 位宽度存储器的时候：HADDR[25:0] →FSMC_A[25:0]。

不论外部接 8 位/16 位宽设备，FSMC_A[0]永远接在外部设备地址 A[0]。这里，TFTLCD 使用的是 16 位数据宽度，所以 HADDR[0]并没有用到，只有 HADDR[25:1]是有效的，对应关系变为：HADDR[25:1]→FSMC_A[24:0]，相当于右移了一位，具体来说，比如地址：0x7E，对应二进制是：01111110，此时 FSMC_A6 是 0 而不是 1，因为要右移一位，这里请特别注意。

另外，HADDR[27:26]的设置，是不需要我们干预的，比如：当你选择使用 Bank1 的第三个区，即使用 FSMC_NE3 来连接外部设备的时候，即对应了 HADDR[27:26]=10，我们要做的就是配置对应第 3 区的寄存器组，来适应外部设备即可。对于 NOR FLASH 控制器，主要是通过 FSMC_BCRx、FSMC_BTRx 和 FSMC_BWTRx 寄存器设置（其中 x=1~4，对应 4 个区）。通过这 3 个寄存器，可以设置 FSMC 访问外部存储器的时序参数，拓宽了可选用的外部存储器的速度范围。FSMC 的 NORFLASH 控制器支持同步和异步突发两种访问方式。选用同步突发访问方式时，FSMC 将 HCLK(系统时钟)分频后，发送给外部存储器作为同步时钟信号 FSMC_CLK。此时需要的设置的时间参数有 2 个：

1，HCLK与FSMC_CLK的分频系数(CLKDIV)，可以为2~16分频；

2，同步突发访问中获得第1个数据所需要的等待延迟(DATLAT)。

对于异步突发访问方式，FSMC 主要设置 3 个时间参数：地址建立时间(ADDSET)、数据建立时间(DATAST)和地址保持时间(ADDHLD)。FSMC 综合了 SRAM / ROM、PSRAM 和 NOR Flash 产品的信号特点，定义了 4 种不同的异步时序模型。选用不同的时序模型时，需要设置不同的时序参数，如表 25.1.3.4 所列：

时序模型		简单描述	时间参数
异步	Mode1	SRAM/CRAM时序	DATAST、ADDSET
	ModeA	SRAM/CRAM OE选通型时序	DATAST、ADDSET
	Mode2/B	NOR FLASH时序	DATAST、ADDSET
	ModeC	NOR FLASH OE选通型时序	DATAST、ADDSET
	ModeD	延长地址保持时间的异步时序	DATAST、ADDSET、ADDHLK
同步突发		根据同步时钟FSMC_CLK读取多个顺序单元的数据	CLKDIV、DATLAT

表25.1.3.4 NOR FLASH控制器支持的时序模型

在实际扩展时，根据选用存储器的特征确定时序模型，从而确定各时间参数与存储器读/写周期参数指标之间的计算关系；利用该计算关系和存储芯片数据手册中给定的参数指标，可计算出 FSMC 所需要的各时间参数，从而对时间参数寄存器进行合理的配置。

模式A支持独立的读写时序控制。这个对我们驱动TFTLCD来说非常有用，因为TFTLCD在读的时候，一般比较慢，而在写的时候可以比较快，如果读写用一样的时序，那么只能以读的时序为基准，从而导致写的速度变慢，或者在读数据的时候，重新配置FSMC的延时，在读操作完成的时候，再配置回写的时序，这样虽然也不会降低写的速度，但是频繁配置，比较麻烦。而如果有独立的读写时序控制，那么我们只要初始化的时候配置好，之后就不用再配置，既可以满足速度要求，又不需要频繁改配置。模式A的写操作及读操作时序分别如图25.1.3.3和图25.1.3.4所示：

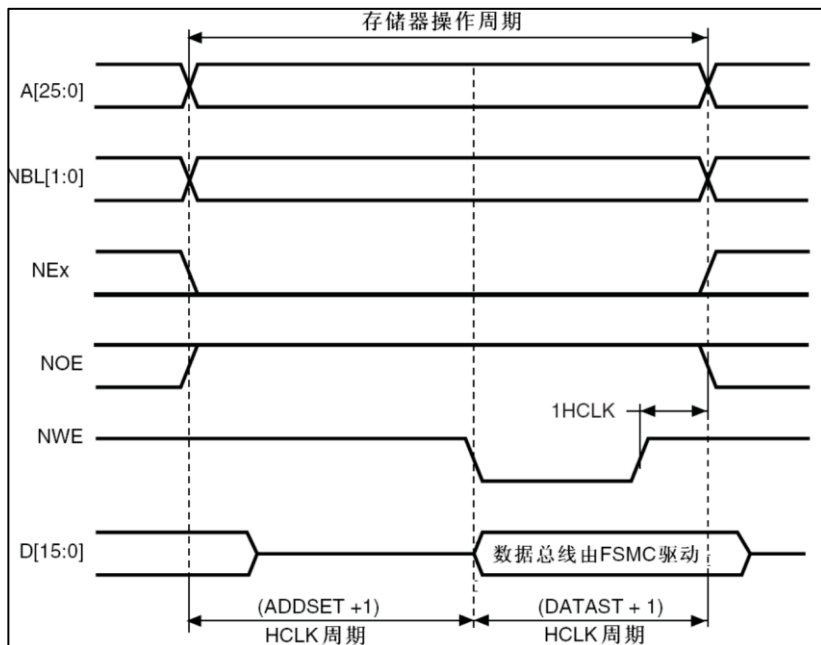


图25.1.3.3 模式A写操作时序

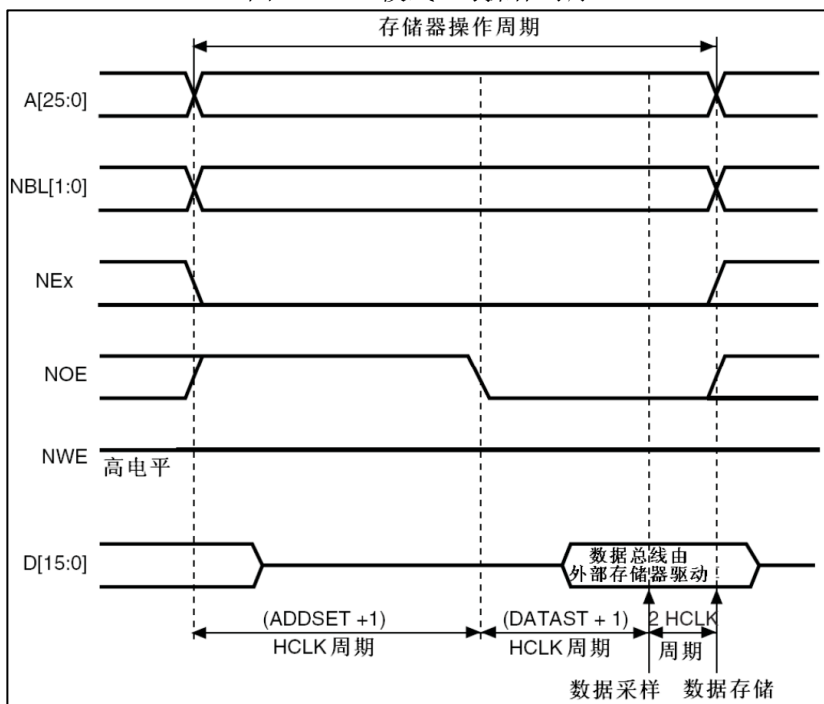


图25.1.3.4 模式A读操作时序图

图 25.1.3.3 和图 25.1.3.4 中的 ADDSET 与 DATAST，是通过不同的寄存器设置的。

以图 25.1.3.3 所示的写操作时序为例，该图表示一个存储器操作周期由地址建立周期(ADDSET)、数据建立周期(DATAST)组成。在地址建立周期中，数据建立周期期间 NWE 信号拉低发出写信号，接着 FSMC 把数据通过数据线传输到存储器中。注意：NEW 拉高后的那 1 个 HCLK 是必要的，以保证数据线上的信号被准确采样。

读操作模式时序类似，区别是它的一个存储器操作周期由地址建立周期(ADDSET)和数据建立周期(DATAST)以及 2 个 HCLK 周期组成，且在数据建立周期期间地址线发出要访问的地址，数据掩码信号线指示出要读取地址的高、低字节部分，片选信号使能存储器芯片；地址建立周期结束后读使能信号线发出读使能信号，接着存储器通过数据信号线把目标数据传输给 FSMC，FSMC 把它交给内核。

当 FSMC 外设被配置成正常工作，并且外部接了 PSRAM，若向 0x60000000 地址写入数据如 0xABCD，FSMC 会自动在各信号线上产生相应的电平信号，写入数据。FSMC 会控制片选信号 NE1 输出低电平，相应的 PSRAM 芯片被片选激活，然后使用地址线 A[25:0]输出 0x60000000，在 NWE 写使能信号线上发出低电平的写使能信号，而要写入的数据信号 0xABCD 则从数据线 D[15:0]输出，然后数据就被保存到 PSRAM 中了。

到这里大家发现没有，之前讲的液晶控制器的 8080 并口模式与 FSMC 接口很像，区别是 FSMC 通过地址访问设备数据，并且可以自动控制相应电平，而 8080 方式则是直接控制，且没有地址线。对比图 25.1.2.2 和图 25.1.3.3，不难发现它们的相似点，我们概括如表 25.1.3.5：

FSMC(NOR/PSRAM)	功能	8080 信号线	功能
FSMC_NEx	片选信号	CSX	片选信号
FSMC_NWR	写使能	WRX	写使能
FSMC_NOE	读使能	RDX	读使能
FSMC_D[15:0]	数据信号	D[17:0]	数据信号
FSMC_A[25:0]	地址信号	D/CX	数据/命令选择

表25.1.3.5 FSMC（NOR/PSRAM）方式和8080并口对比

如果能用某种方式把 FSMC 的地址线和 8080 方式下的等效起来，那不就可以直接用 FSMC 等效 8080 方式操作 LCD 屏的显存了？FSMC 利用地址线访问数据，并自动设置地址线和相关控制信号线的电平，如果我们对命令操作和数据操作采用不同的地址来访问，同时使得操作数据时，地址线上的一个引脚的电平为高，操作命令时，同一个引脚的电平为低的话，就可以完美解决这个问题了！

战舰 STM32 开发板把 TFT-LCD 就是用的 FSMC_NE4 做片选，把 RS 连接在 A10 上面的。我们来分析一下要实现上面的通过地址自动切换命令和数据的实现方式。

首先 NOR/PSRAM 储块地址范围：0x6000 0000 ~ 0x6FFF FFFF，基地址是 0x6000 0000，每个存储块是 64MB，那么这时候我们访问 LCD 的地址应该是第 4 个存储块，编号从 1 开始，访问 LCD 的起始地址就是 $0x6000\ 0000 + (0x400\ 0000 * (x - 1)) = 0x6C00\ 0000$ ，即从 0x6C00 0000 起的 64MB 内存地址都可以去访问 LCD。

FSMC_A10 对应地址值： $2^{10} * 2 = 0x800$ （16 位模式时，参考表 25.1.3.3 及之后对 HADDR 和 FSMC 地址线对应关系的描述：HADDR[25:1]→FSMC_A[24:0]，所以这里计算时还需要乘 2）；则写命令时的地址为： $0x6C00\ 0000 + 2^{10} * 2 = 0x6C00\ 0800$ 。写数据的地址就是使 FSMC_A10 为 0 的其它任意地址。（大家不要被地址访问的思路带进去了，以为接下来就是用 FSMC 的地址偏移来操作显存了，实际显存的操作还是归 MCU 屏管理。我们使能了 FSMC 功能后，就可以直接在我们设置的地址读写数据。实际上我们只用到了两个固定的地址：一个地址把 FSMC_A10 位置 1，另一个把该位置 0，但要保证这两个地址在各个 BANK 的管理范围内）。

STM32F1 的 FSMC 支持 8/16 位数据宽度，我们这里用到的 LCD 是 16 位宽度的，所以在设置的时候，选择 16 位宽就 OK 了。向这两个地址写的 16 进制数据会被直接送到数据线上，根据地址自动解析为命令或者数据，通过这样一个过程，我们就完成了用 FSMC 模拟 8080 并口的操作，最终完成对液晶控制器的控制。

25.1.4 FSMC 关联寄存器简介

接下来我们讲解一下 Bank1 的几个控制寄存器。

首先，我们介绍 SRAM/NOR 闪存片选控制寄存器：FSMC_BCRx（x=1~4），该寄存器各位描述如图 25.1.4.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
保留												CBURSTRW	保留				EXTMOD	WAITEN	WREN	WAITCFG	WRAPMOD	WAITPOL	BURSTEN	保留	FACCEN	MWID		MTYP		MUXEN	MBKEN			
res												rw	res				rw	rw	rw	rw	rw	rw	rw	rw	res	rw	rw		rw		rw	rw	rw	rw
位14		EXTMOD: 扩展模式使能 (Extended mode enable) 该位允许FSMC使用FSMC_BWTR寄存器，即允许读和写使用不同的时序。 0：不使用FSMC_BWTR寄存器，这是复位后的默认状态。 1：FSMC使用FSMC_BWTR寄存器。																																
位12		WREN: 写使能位 (Write enable bit) 该位指示FSMC是否允许/禁止对存储器的写操作。 0：禁止FSMC对存储器的写操作，否则产生一个AHB错误。 1：允许FSMC对存储器的写操作；这是复位后的默认状态。																																
位5:4		MWID: 存储器数据总线宽度 (Memory databus width) 定义外部存储器总线的宽度，适用于所有类型的存储器。 00：8位， 01：16位(复位后的默认状态)， 10：保留，不能用 11：保留，不能用																																
位3:2		MTYP: 存储器类型 (Memory type) 定义外部存储器的类型： 00：SRAM、ROM(存储器块2...4在复位后的默认值) 01：PSRAM(Cellular RAM: CRAM) 10：NOR闪存(存储器块1在复位后的默认值) 11：保留																																
位0		MBKEN: 存储器块使能位 (Memory bank enable bit) 开启对应的存储器块。复位后存储器块1是开启的，其它所有存储器块为禁用。访问一个禁用的存储器块将在AHB总线上产生一个错误。 0：禁用对应的存储器块。 1：启用对应的存储器块。																																

图25.1.4.1 FSMC_BCRx寄存器各位描述

该寄存器我们在本章用到的设置有: EXTMOD、WREN、MWID、MTYP 和 MBKEN 这几个设置, 我们将逐个介绍。

EXTMOD: 扩展模式使能位, 也就是是否允许读写不同的时序, 很明显, 我们本章需要读写不同的时序, 故该位需要设置为 1。

WREN: 写使能位。我们需要向 TFTLCD 写数据, 故该位必须设置为 1。

MWID[1:0]: 存储器数据总线宽度。我们的 TFTLCD 是 16 位数据线, 所以设置该值为 01。

MTYP[1:0]: 存储器类型。前面提到, 我们把 TFTLCD 当成 SRAM 用, 所以需要设置该值为 00。

MBKEN: 存储块使能位。这个容易理解, 我们需要用到该存储块控制 TFTLCD, 当然要使能这个存储块了。

接下来, 我们看看 SRAM/NOR 闪存片选时序寄存器: FSMC_BTRx (x=1~4), 该寄存器各位描述如图 25.1.4.2 所示:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
保留		ACCMOD		DATLAT				CLKDIV				BUSTURN				DATAST								ADDHLD				ADDSET									
res		rw		rw				rw				rw				rw								rw				rw									
		位29:28		ACCMOD: 访问模式 (Access mode) 定义异步访问模式。这2位只在FSMC_BCRx寄存器的EXTMOD位为1时起作用。 00: 访问模式A 01: 访问模式B 10: 访问模式C 11: 访问模式D																																	
		位15:8		DATAST: 数据保持时间 (Data-phase duration) 这些位定义数据的保持时间(见图162至图174), 适用于SRAM、ROM和异步总线复用模式的NOR闪存操作。 0000 0000: 保留 0000 0001: DATAST保持时间=2个HCLK时钟周期 0000 0010: DATAST保持时间=3个HCLK时钟周期 1111 1111: DATAST保持时间=256个HCLK时钟周期(这是复位后的默认数值)。 对于每一种存储器类型和访问方式的数据保持时间, 请参考对应的图表(见图162至图174)。 例如: 模式1、读操作、DATAST=1: 数据保持时间=DATAST+3=4个HCLK时钟周期。																																	
		位3:0		ADDSET: 地址建立时间 (Address setup phase duration) 这些位定义地址的建立时间(见图162至图174), 适用于SRAM、ROM和异步总线复用模式的NOR闪存操作。 0000: ADDSET建立时间=1个HCLK时钟周期 1111: ADDSET建立时间=16个HCLK时钟周期(这是复位后的默认数值)。 对于每一种存储器类型和访问方式的地址建立时间, 请参考对应的图表(见图162至图174)。 例如: 模式2、读操作、ADDSET=1: 地址建立时间=ADDSET+1=2个HCLK时钟周期 注: 在同步操作中, 这个参数不起作用, 地址建立时间始终是1个存储器时钟周期。																																	

图25.1.4.2 FSMC_BTRx寄存器各位描述

这个寄存器包含了每个存储器块的控制信息，可以用于SRAM、ROM和NOR闪存存储器。如果FSMC_BCRx寄存器中设置了EXTMOD位，则有两个时序寄存器分别对应读(本寄存器)和写操作(FSMC_BWTRx寄存器)。因为我们要求读写分开时序控制，所以EXTMOD是使能了的，也就是本寄存器是读操作时序寄存器，控制读操作的相关时序。本章我们要用到的设置有：ACCMOD、DATAST和ADDSET这三个设置。

ACCMOD[1:0]：访问模式。本章我们用到模式A，故设置为00。

DATAST[7:0]：数据保持时间。对 ILI9341 来说，其实就是 RD 低电平持续时间，一般为 355ns。而一个 HCLK 时钟周期为 13.9ns 左右 (1/72Mhz)，为了兼容其他屏，我们这里设置 DATAST 为 15，也就是 16 个 HCLK 周期，时间大约是 222ns (未计算数据存储的 2 个 HCLK 时间，对 9341 来说超频了，但是实际上是可以正常使用的)。

ADDSET[3:0]：地址建立时间。其建立时间为：ADDSET 个 HCLK 周期，最大为 15 个 HCLK 周期。对 ILI9341 来说，这里相当于 RD 高电平持续时间，为 90ns，本来这里我们应该设置和 DATAST 一样，但是由于 STM32F103 FSMC 的性能问题，就算设置 ADDSET 为 0，RD 的高电平持续时间也超过 90ns。所有，我们这里可以设置 ADDSET 为较小的值，本章我们设置 ADDSET 为 1，即 2 个 HCLK 周期。

最后，我们再来看看SRAM/NOR闪写时序寄存器：FSMC_BWTRx (x=1~4)，该寄存器各位描述如图25.1.4.3所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留		ACCMOD		DATLAT				CLKDIV				保留				DATAST						ADDHLD				ADDSET					
res		rw		rw				rw				res				rw						rw				rw					
		位29:28		ACCMOD: 访问模式 (Access mode) 定义异步访问模式。这2位只在FSMC_BCRx寄存器的EXTMOD位为1时起作用。 00: 访问模式A 01: 访问模式B 10: 访问模式C 11: 访问模式D																											
		位15:8		DATAST: 数据保持时间 (Data-phase duration) 这些位定义数据的保持时间(见图162至图174)，适用于SRAM、ROM和异步总线复用模式的NOR闪存操作。 0000 0000: 保留 0000 0001: DATAST保持时间=2个HCLK时钟周期 0000 0010: DATAST保持时间=3个HCLK时钟周期 1111 1111: DATAST保持时间=256个HCLK时钟周期(这是复位后的默认数值)。																											
		位3:0		ADDSET: 地址建立时间 (Address setup phase duration) 这些位以HCLK周期数定义地址的建立时间(见图162至图174)，适用于SRAM、ROM和异步总线复用模式的NOR闪存操作。 0000: ADDSET建立时间=1个HCLK时钟周期 1111: ADDSET建立时间=16个HCLK时钟周期(这是复位后的默认数值)。 注：在同步NOR闪存操作中，这个参数不起作用，地址建立时间始终是1个闪存时钟周期。																											

图25.1.4.3 FSMC_BWTRx寄存器各位描述

该寄存器在本章用作写操作时序控制寄存器，需要用到的设置同样是：ACCMOD、DATAST 和 ADDSET 这三个设置。这三个设置的方法同 FSMC_BTRx 一模一样，只是这里对应的是写操作的时序，ACCMOD 设置同 FSMC_BTRx 一模一样，同样是选择模式 A，另外 DATAST 和 ADDSET 则对应低电平和高电平持续时间，对 ILI9341 来说，这两个时间只需要 15ns 就够了，比读操作快得多。所以我们这里设置 DATAST 为 1，即 2 个 HCLK 周期，时间约为 28ns。然后 ADDSET（也存在性能问题）设置为 0，即 1 个 HCLK 周期，实际 WR 高电平时间就可以满足。

至此，我们对 STM32F1 的 FSMC 介绍就差不多了，通过以上两个小节的了解，我们可以开始写 LCD 的驱动代码了。不过，这里还要给大家做下科普，在 MDK 的寄存器定义里面，并没有定义 FSMC_BCRx、FSMC_BTRx、FSMC_BWTRx 等这个单独的寄存器，而是将他们进行了一些组合。

FSMC_BCRx 和 FSMC_BTRx，组合成 BTCR[8]寄存器组，他们的对应关系如下：

BTCR[0]对应 FSMC_BCR1，

BTCR[1]对应 FSMC_BTR1

BTCR[2]对应 FSMC_BCR2

BTCR[3]对应 FSMC_BTR2

BTCR[4]对应 FSMC_BCR3

BTCR[5]对应 FSMC_BTR3

BTCR[6]对应 FSMC_BCR4

BTCR[7]对应 FSMC_BTR4

FSMC_BWTRx 则组合成 BWTR[7]，他们的对应关系如下：

BWTR[0]对应 FSMC_BWTR1，

BWTR[2]对应 FSMC_BWTR2，

BWTR[4]对应 FSMC_BWTR3，

BWTR[6]对应 FSMC_BWTR4，

BWTR[1]、BWTR[3]和 BWTR[5]保留，没有用到。

通过上面的讲解，通过对 FSMC 相关的寄存器的描述，大家对 FSMC 的原理有了一个初步

这些线的连接，开发板的内部已经连接好了，我们只需要将 TFTLCD 模块插上去就好了。

需要说明的是，开发板上设计的 TFT-LCD 模块插座，已经把模块模块的 RST 信号线直接接到我们开发板的复位脚上，所以不需要软件控制，这样可以省下来一个 IO 口。另外我们还需要一个背光控制线来控制 LCD 的背光灯，因为 LCD 不会自发光，没有背光灯的情况下我们是看不到 LCD 上显示的内容的。所以，我们总共需要的 IO 口数目为 22 个。

25.3 程序设计

25.3.1 FSMC 和 SRAM 的 HAL 库驱动

SRAM 和 FMC 在 HAL 库中的驱动代码在 `stm32f1xx_ll_fsmc.c/stm32f1xx_hal_sram.c` 以及 `stm32f1xx_ll_fsmc.h/stm32f1xx_hal_sram.h` 中。

1. HAL_SRAM_Init 函数

SRAM 的初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_SRAM_Init(SRAM_HandleTypeDef *hsram,
                                FSMC_NORSRAM_TimingTypeDef *Timing, FSMC_NORSRAM_TimingTypeDef *ExtTiming);
```

● 函数描述：

用于初始化 SRAM，注意这个函数不限制一定是 SRAM，只要时序类似，均可使用。前面说过，这里我们把 LCD 当作 SRAM 使用，因为他们时序类似。

● 函数形参：

形参 1 是 `SRAM_HandleTypeDef` 结构体类型指针变量，其定义如下：

```
typedef struct
{
    FSMC_NORSRAM_TypeDef *Instance;          /* 寄存器基地址 */
    FSMC_NORSRAM_EXTENDED_TypeDef *Extended; /* 扩展模式寄存器基地址 */
    FSMC_NORSRAM_InitTypeDef Init;           /* SRAM 初始化结构体 */
    HAL_LockTypeDef Lock;                    /* SRAM 锁对象结构体 */
    __IO HAL_SRAM_StateTypeDef State;        /* SRAM 设备访问状态 */
    DMA_HandleTypeDef *hdma;                /* DMA 结构体 */
} SRAM_HandleTypeDef;
```

1) `Instance`：指向 FSMC 寄存器基地址。我们直接写 `FSMC_NORSRAM_DEVICE` 即可，因为 HAL 库定义好了宏定义 `FSMC_NORSRAM_DEVICE`，也就是如果是 SRAM 设备，直接填写这个宏定义标识符即可。

2) `Extended`：指向 FSMC 扩展模式寄存器基地址，因为我们要配置的读写时序是不一样的。前面讲的 `FSMC_BCRx` 寄存器的 `EXTMOD` 位，我们会配置为 1 允许读写不同的时序，所以还要指定写操作时序寄存器地址，也就是通过参数 `Extended` 来指定的，这里设置为 `FSMC_NORSRAM_EXTENDED_DEVICE`。

3) `Init`：用于对 FSMC 的初始化配置，这个比较重要，后面再来讲解。

4) `Lock`：用于配置锁状态。

5) `State`：SRAM 设备访问状态。

6) `hdma`：在使用 DMA 时候才使用，这里就先不讲解了。

成员变量 `Init` 是 `FSMC_NORSRAM_InitTypeDef` 结构体指针类型，该变量才是真正用来设置 SRAM 控制接口参数的。下面详细了解这个结构体定义：

```
typedef struct
{
    uint32_t NSBank;          /* 存储区块号 */
    uint32_t DataAddressMux;  /* 地址/数据复用使能 */
    uint32_t MemoryType;      /* 存储器类型 */
    uint32_t MemoryDataWidth; /* 存储器数据宽度 */
    uint32_t BurstAccessMode; /* 突发模式配置 */
    uint32_t WaitSignalPolarity; /* 设置等待信号的极性 */
    uint32_t WrapMode;        /* 突发下存储器传输使能 */
    uint32_t WaitSignalActive; /* 等待状态之前或等待状态期间 */
    uint32_t WriteOperation;  /* 存储器写使能 */
}
```

```
uint32_t WaitSignal;          /* 使能或者禁止通过等待信号来插入等待状态 */
uint32_t ExtendedMode;        /* 使能或者禁止使能扩展模式 */
uint32_t AsynchronousWait;    /* 用于异步传输期间, 使能或者禁止等待信号 */
uint32_t WriteBurst;          /* 用于使能或者禁止异步的写突发操作 */
uint32_t PageSize;            /* 设置页大小 */
}FSMC_NORSRAM_InitTypeDef;
```

NSBank 用来指定使用到的存储块区号, 我们硬件设计时使用的存储块区号 4, 所以选择值为 FSMC_NORSRAM_BANK4。

DataAddressMux 用来设置是否使能地址/数据复用, 该变量仅对 NOR/PSRAM 有效, 所以这里我们选择不使能地址/数据复用值 FSMC_DATA_ADDRESS_MUX_DISABLE 即可。

MemoryType 用来设置存储器类型, 这里我们把 LCD 当 SRAM 使用, 所以设置为 FSMC_MEMORY_TYPE_SRAM 即可。

MemoryDataWidth 用来设置存储器数据总线宽度, 可选 8 位还是 16 位, 这里我们选择 16 位数据宽度 FSMC_NORSRAM_MEM_BUS_WIDTH_16。

WriteOperation 用来设置存储器写使能, 也就是是否允许写入。毫无疑问我们会进行存储器写操作, 所以这里设置为 FSMC_WRITE_OPERATION_ENABLE。

ExtendedMode 用来设置是否使能扩展模式, 也就是是否允许读写使用不同时序, 前面讲解过本实验读写采用不同时序, 所以设置值为使能值 FSMC_EXTENDED_MODE_ENABLE。

其他参数 WriteBurst, BurstAccessMode, WaitSignalPolarity, WaitSignalActive, WaitSignal, AsynchronousWait 等是用于在突发访问和异步时序情况下, 这里我们不做过多讲解。

形参 2 Timing 和形参 3 ExtTiming 都是 FSMC_NORSRAM_TimingTypeDef 结构体类型指针变量, 其定义如下:

```
typedef struct
{
    uint32_t AddressSetupTime;          /* 地址建立时间 */
    uint32_t AddressHoldTime;           /* 地址保持时间 */
    uint32_t DataSetupTime;             /* 数据建立时间 */
    uint32_t BusTurnAroundDuration;     /* 总线周转阶段的持续时间 */
    uint32_t CLKDivision;               /* CLK 时钟输出信号的周期 */
    uint32_t DataLatency;               /* 同步突发 NOR FLASH 的数据延迟 */
    uint32_t AccessMode;                /* 异步模式配置 */
}FSMC_NORSRAM_TimingTypeDef;
```

对于本实验, 读速度比写速度慢得多, 因此读写时序不一样, 所以对于 Timing 和 ExtTiming 要设置了不同的值, 其中 Timing 设置写时序参数, ExtTiming 设置读时序参数。

下面解析一下结构体的成员变量:

AddressSetupTime 用来设置地址建立时间, 可以理解为 RD/WR 的高电平时间。

AddressHoldTime 用来设置地址保持时间, 模式 A 并没有用到。

DataSetupTime 用来设置数据建立时间, 可以理解为 RD/WR 的低电平时间。

BusTurnAroundDuration 用来配置总线周转阶段的持续时间, NOR FLASH 用到。

CLKDivision 用来配置 CLK 时钟输出信号的周期, 以 HCLK 周期数表示。若控制异步存储器, 该参数无效。

DataLatency 用来设置同步突发 NOR FLASH 的数据延迟。若控制异步存储器, 该参数无效。

AccessMode 用来设置异步模式, HAL 库允许其取值范围为 FSMC_ACCESS_MODE_A、FSMC_ACCESS_MODE_B、FSMC_ACCESS_MODE_C 和 FSMC_ACCESS_MODE_D, 这里我们用是异步模式 A, 所以取值为 FSMC_ACCESS_MODE_A。

● 函数返回值:

HAL_StatusTypeDef 枚举类型的值。

● 注意事项:

和其他外设一样, HAL 库也提供了 SRAM 的初始化 MSP 回调函数, 函数声明如下:

```
void HAL_SRAM_MspInit(SRAM_HandleTypeDef *hsram);
```

2. FSMC_NORSRAM_Extended_Timing_Init 函数

FSMC_NORSRAM_Extended_Timing_Init 函数是初始化扩展时序模式函数。其声明如下:

```
HAL_StatusTypeDef FSMC_NORSRAM_Extended_Timing_Init(
    FSMC_NORSRAM_EXTENDED_TypeDef *Device, FSMC_NORSRAM_TimingTypeDef *Timing,
    uint32_t Bank, uint32_t ExtendedMode);
```

- **函数描述：**
该函数用于初始化扩展时序模式。
- **函数形参：**
形参 1 是 FSMC_NORSRAM_EXTENDED_TypeDef 结构体类型指针变量，扩展模式寄存器基地址选择。
形参 2 是 FSMC_NORSRAM_TimingTypeDef 结构体类型指针变量，可以是读或者写时序结构体。
形参 3 是储存区块号。
形参 4 是使能或者禁止扩展模式。
- **函数返回值：**
HAL_StatusTypeDef 枚举类型的值。
- **注意事项：**
该函数我们用于重新配置写或者读时序。

FSMC 驱动 LCD 显示配置步骤

1) 使能 FSMC 和相关 GPIO 时钟，并设置好 GPIO 工作模式

我们通过 FSMC 控制 LCD，所以先需要使能 FSMC 以及相关 GPIO 口的时钟，并设置好 GPIO 的工作模式。

2) 设置 FSMC 参数

这里我们需要设置 FSMC 的相关访问参数（数据位宽、访问时序、工作模式等），以匹配液晶驱动 IC，这里我们通过 HAL_SRAM_Init 函数完成 FSMC 参数配置，详见本例程源码。

3) 初始化 LCD

由于我们例程兼容了很多种液晶驱动 IC，所以先要读取对应 IC 的驱动型号，然后根据不同的 IC 型号来调用不同的初始化函数，完成对 LCD 的初始化。

注意：这些初始化函数里面的代码，都是由 LCD 厂家提供，一般不需要改动，也不需要深究，我们直接照抄即可。

4) 实现 LCD 画点&读点函数

在初始化 LCD 完成以后，我们就可以控制 LCD 显示了，而最核心的一个函数，就是画点和读点函数，只要实现这两个函数，后续的各种 LCD 操作函数，都可以基于这两个函数实现。

5) 实现其他 LCD 操作函数

在完成画点和读点两个最基础的 LCD 操作函数以后，我们就可以基于这两个函数实现各种 LCD 操作函数了，比如画线、画矩形、显示字符、显示字符串、显示数字等，如果不够用还可以根据自己需要来添加。详见本例程源码。

25.3.2 程序流程图

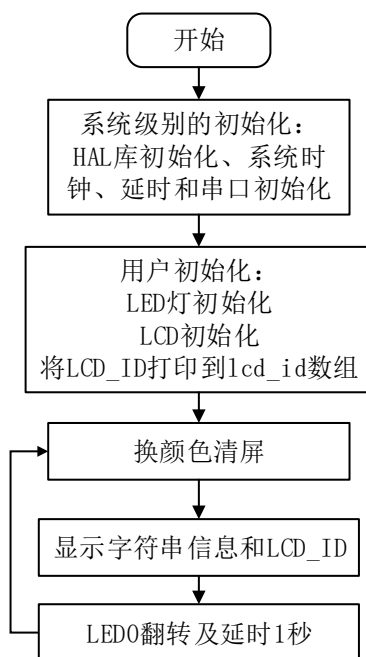


图 25.3.2.1 TFTLCD (MCU 屏) 实验程序流程图

25.3.2 程序解析

1. LCD 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。液晶（LCD）驱动源码包括四个文件：lcd.c、lcd.h、lcd_ex.c 和 lcdfont.h。

lcd.c 和 lcd.h 文件是驱动函数和引脚接口宏定义以及函数声明等。lcd_ex.c 存放各个 LCD 驱动 IC 的寄存器初始化部分代码，是 lcd.c 文件的补充文件，起到简化 lcd.c 文件的作用。lcdfont.h 头文件存放了 4 种字体大小不一样的 ASCII 字符集（12*12、16*16、24*24 和 32*32）。这个跟 oledfont.h 头文件一样的，只是这里多了 32*32 的 ASCII 字符集，制作方法请回顾 OLED 实验。

下面我们还是先介绍 lcd.h 文件，首先是 LCD 的引脚定义：

```

/* LCD RST/WR/RD/BL/CS/RS 引脚 定义
 * LCD_D0~D15, 由于引脚太多, 就不在这里定义了, 直接在 lcd_init 里面修改. 所以在移植的时候, 除了
 * 改这 6 个 IO 口, 还得改 LCD_Init 里面的 D0~D15 所在的 IO 口.
 */

/* RESET 和系统复位脚共用 所以这里不用定义 RESET 引脚 */

#define LCD_WR_GPIO_PORT      GPIOD
#define LCD_WR_GPIO_PIN      GPIO_PIN_5
#define LCD_WR_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOD_CLK_ENABLE(); }while(0)

#define LCD_RD_GPIO_PORT      GPIOD
#define LCD_RD_GPIO_PIN      GPIO_PIN_4
#define LCD_RD_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOD_CLK_ENABLE(); }while(0)

#define LCD_BL_GPIO_PORT      GPIOB
#define LCD_BL_GPIO_PIN      GPIO_PIN_0
#define LCD_BL_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)

/* LCD_CS (需要根据 LCD_FSMC_NEX 设置正确的 IO 口) 和 LCD_RS (需要根据 LCD_FSMC_AX 设置正确的
IO 口) 引脚 定义 */
#define LCD_CS_GPIO_PORT      GPIOG
#define LCD_CS_GPIO_PIN      GPIO_PIN_12
  
```

```
#define LCD_CS_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIO_CLK_ENABLE();}while(0)

#define LCD_RS_GPIO_PORT          GPIOG
#define LCD_RS_GPIO_PIN          GPIO_PIN_0
#define LCD_RS_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOG_CLK_ENABLE();}while(0)
```

第一部分的宏定义是 LCD WR/RD/BL/CS/RS/DATA 引脚定义,需要注意的是:LCD 的 RST 引脚和系统复位脚连接在一起,所以不用单独使用一个 IO 口(节省一个 IO 口)。而 DATA 引脚直接用的是 FSMC_D[x]引脚,具体可以查看前面的描述。

下面介绍我们在 lcd.h 里面定义的一个重要的结构体:

```
/* LCD 重要参数集 */
typedef struct
{
    uint16_t width;        /* LCD 宽度 */
    uint16_t height;       /* LCD 高度 */
    uint16_t id;           /* LCD ID */
    uint8_t dir;           /* 横屏还是竖屏控制: 0, 竖屏; 1, 横屏。 */
    uint16_t wramcmd;       /* 开始写 gram 指令 */
    uint16_t setxcmd;       /* 设置 x 坐标指令 */
    uint16_t setycmd;       /* 设置 y 坐标指令 */
} _lcd_dev;
extern _lcd_dev lcddev; /* 管理 LCD 重要参数 */
/* LCD 的画笔颜色和背景色 */
extern uint32_t g_point_color; /* 默认红色 */
extern uint32_t g_back_color; /* 背景颜色,默认为白色 */
```

该结构体用于保存一些 LCD 重要参数信息,比如 LCD 的长宽、LCD ID(驱动 IC 型号)、LCD 横竖屏状态等,这个结构体虽然占用了十几个字节的内存,但是却可以让我们的驱动函数支持不同尺寸的 LCD,同时可以实现 LCD 横竖屏切换等重要功能,所以还是利大于弊的。最后声明 _lcd_dev 结构体类型变量 lcddev, lcddev 在 lcd.c 中定义。

紧接着就是 g_point_color 和 g_back_color 变量的声明,它们也是在 lcd.c 中被定义。g_point_color 变量用于保存 LCD 的画笔颜色, g_back_color 则是保存 LCD 的背景色。

下面是 LCD 背光控制 IO 口的宏定义:

```
/* LCD 背光控制 */
#define LCD_BL(x) do{ x ? \
    HAL_GPIO_WritePin(LCD_BL_GPIO_PORT, LCD_BL_GPIO_PIN, GPIO_PIN_SET) : \
    HAL_GPIO_WritePin(LCD_BL_GPIO_PORT, LCD_BL_GPIO_PIN, GPIO_PIN_RESET); \
}while(0)
```

本实验,我们用到 FSMC 驱动 LCD,通过前面的介绍,我们知道 TFTLCD 的 RS 接在 FSMC 的 A10 上面, CS 接在 FSMC_NE4 上,并且是 16 位数据总线。即我们使用的是 FSMC 存储器 1 的第 4 区,我们定义如下 LCD 操作结构体(在 lcd.h 里面定义):

```
/* LCD 地址结构体 */
typedef struct
{
    volatile uint16_t LCD_REG;
    volatile uint16_t LCD_RAM;
} LCD_TypeDef;

/* LCD_BASE 的详细解算方法:
* 我们一般使用 FSMC 的块 1 (BANK1) 来驱动 TFTLCD 液晶屏 (MCU 屏), 块 1 地址范围总大小为 256MB,
均分成 4 块:
* 存储块 1 (FSMC_NE1) 地址范围: 0X6000 0000 ~ 0X63FF FFFF
* 存储块 2 (FSMC_NE2) 地址范围: 0X6400 0000 ~ 0X67FF FFFF
* 存储块 3 (FSMC_NE3) 地址范围: 0X6800 0000 ~ 0X6BFF FFFF
* 存储块 4 (FSMC_NE4) 地址范围: 0X6C00 0000 ~ 0X6FFF FFFF
*
* 我们需要根据硬件连接方式选择合适的片选 (连接 LCD_CS) 和地址线 (连接 LCD_RS)
* 战舰 F103 开发板使用 FSMC_NE4 连接 LCD_CS, FSMC_A10 连接 LCD_RS, 16 位数据线, 计算方法如下:
* 首先 FSMC_NE4 的基地址为: 0X6C00 0000; NEx 的基址为 (x=1/2/3/4): 0X6000 0000 +
(0X400 0000 * (x - 1))
```

```

* FSMC_A10 对应地址值:  $2^{10} * 2 = 0X800$ ; FSMC_Ay 对应的地址为 ( $y = 0 \sim 25$ ):  $2^y * 2$ 
*
* LCD->LCD_REG, 对应 LCD_RS = 0 (LCD 寄存器); LCD->LCD_RAM, 对应 LCD_RS = 1 (LCD 数据)
* 则 LCD->LCD_RAM 的地址为:  $0X6C00\ 0000 + 2^{10} * 2 = 0X6C00\ 0800$ 
* LCD->LCD_REG 的地址可以为 LCD->LCD_RAM 之外的任意地址。
* 由于我们使用结构体管理 LCD_REG 和 LCD_RAM (REG 在前, RAM 在后, 均为 16 位数据宽度)
* 因此 结构体的基地址 (LCD_BASE) = LCD_RAM - 2 =  $0X6C00\ 0800 - 2$ 
*
* 更加通用的计算公式为 ((片选脚 FSMC_NEx)  $x=1/2/3/4$ , (RS 接地址线 FSMC_Ay)  $y=0\sim25$ ):
* LCD_BASE =  $(0X6000\ 0000 + (0X400\ 0000 * (x - 1))) | (2^y * 2 - 2)$ 
* 等效于 (使用移位操作)
* LCD_BASE =  $(0X6000\ 0000 + (0X400\ 0000 * (x - 1))) | ((1 << y) * 2 - 2)$ 
*/
#define LCD_BASE (uint32_t)((0X60000000 + (0X4000000 * (LCD_FSMC_NEX - 1))) | ((1 << LCD_FSMC_AX) * 2) - 2)
#define LCD ((LCD_TypeDef *) LCD_BASE)

```

其中 LCD_BASE, 必须根据我们外部电路的连接来确定, 我们使用 BANK1 的存储块 4 的寻址范围为 0X6C000000~6FFFFFFF, 我们需要在这个地址范围内找到两个地址, 实现对 RS 位 (FSMC_A10 位) 的 0 和 1 的控制。这两个地址的取值方法, 我们在前面的 25.1.3 的末尾已经详细说明了。为了方便控制和节省内存, 我们使这两个地址变成相邻的两个 16 进制指针, 这样就可以用前面定义的 LCD_TypeDef 来管理这两个地址了。

根据我们的算法和定义, 我们将这个地址强制转换为 LCD_TypeDef 结构体地址, 那么可以得到 LCD->LCD_REG 的地址就是 0X6C00 07FE, 对应 A10 的状态为 0 (即 RS=0), 而 LCD->LCD_RAM 的地址就是 0X6C00 0800 (结构体地址自增), 对应 A10 的状态为 1 (即 RS=1)。

所以, 有了这个定义, 当我们要往 LCD 写命令/数据的时候, 可以这样写:

```

LCD->LCD_REG = CMD; /* 写命令 */
LCD->LCD_RAM = DATA; /* 写数据 */

```

而读的时候反过来操作就可以了, 如下所示:

```

CMD = LCD->LCD_REG; /* 读 LCD 寄存器 */
DATA = LCD->LCD_RAM; /* 读 LCD 数据 */

```

这其中, CS、WR、RD 和 IO 口方向都是由 FSMC 硬件自动控制, 不需要我们手动设置了。

最后是一些其他的宏定义, 包括 LCD 扫描方向和颜色, 以及 SSD1963 相关配置参数。

下面开始对 lcd.c 文件介绍, 先看 LCD 初始化函数, 其定义如下:

```

/**
 * @brief      初始化 LCD
 * @note      该初始化函数可以初始化各种型号的 LCD (详见本.c 文件最前面的描述)
 *
 * @param      无
 * @retval     无
 */
void lcd_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;
    FSMC_NORSRAM_TimingTypeDef fsmc_read_handle;
    FSMC_NORSRAM_TimingTypeDef fsmc_write_handle;

    LCD_CS_GPIO_CLK_ENABLE(); /* LCD_CS 脚时钟使能 */
    LCD_WR_GPIO_CLK_ENABLE(); /* LCD_WR 脚时钟使能 */
    LCD_RD_GPIO_CLK_ENABLE(); /* LCD_RD 脚时钟使能 */
    LCD_RS_GPIO_CLK_ENABLE(); /* LCD_RS 脚时钟使能 */
    LCD_BL_GPIO_CLK_ENABLE(); /* LCD_BL 脚时钟使能 */

    gpio_init_struct.Pin = LCD_CS_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP; /* 推挽复用 */
    gpio_init_struct.Pull = GPIO_PULLUP; /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH; /* 高速 */
    HAL_GPIO_Init(LCD_CS_GPIO_PORT, &gpio_init_struct); /* 初始化 LCD_CS 引脚 */
}

```

```

gpio_init_struct.Pin = LCD_WR_GPIO_PIN;
HAL_GPIO_Init(LCD_WR_GPIO_PORT, &gpio_init_struct);    /* 初始化 LCD_WR 引脚 */

gpio_init_struct.Pin = LCD_RD_GPIO_PIN;
HAL_GPIO_Init(LCD_RD_GPIO_PORT, &gpio_init_struct);    /* 初始化 LCD_RD 引脚 */

gpio_init_struct.Pin = LCD_RS_GPIO_PIN;
HAL_GPIO_Init(LCD_RS_GPIO_PORT, &gpio_init_struct);    /* 初始化 LCD_RS 引脚 */

gpio_init_struct.Pin = LCD_BL_GPIO_PIN;
gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;
HAL_GPIO_Init(LCD_BL_GPIO_PORT, &gpio_init_struct);    /* 初始化 LCD_BL 引脚 */

g_sram_handle.Instance = FSMC_NORSRAM_DEVICE;
g_sram_handle.Extended = FSMC_NORSRAM_EXTENDED_DEVICE;

g_sram_handle.Init.NSBank = FSMC_NORSRAM_BANK4;        /* 使用 NE4 */
/*地址/数据线不复用*/
g_sram_handle.Init.DataAddressMux = FSMC_DATA_ADDRESS_MUX_DISABLE;
/*16 位数据宽度*/
g_sram_handle.Init.MemoryDataWidth = FSMC_NORSRAM_MEM_BUS_WIDTH_16;
/*是否使能突发访问,仅对同步突发存储器有效,此处未用到*/
g_sram_handle.Init.BurstAccessMode = FSMC_BURST_ACCESS_MODE_DISABLE;
/*等待信号的极性,仅在突发模式访问下有用*/
g_sram_handle.Init.WaitSignalPolarity = FSMC_WAIT_SIGNAL_POLARITY_LOW;
/* 存储器是在等待周期之前的一个时钟周期还是等待周期期间使能 NWAIT */
g_sram_handle.Init.WaitSignalActive = FSMC_WAIT_TIMING_BEFORE_WS;
/* 存储器写使能 */
g_sram_handle.Init.WriteOperation = FSMC_WRITE_OPERATION_ENABLE;
/* 等待使能位,此处未用到 */
g_sram_handle.Init.WaitSignal = FSMC_WAIT_SIGNAL_DISABLE;
/* 读写使用不同的时序 */
g_sram_handle.Init.ExtendedMode = FSMC_EXTENDED_MODE_ENABLE;
/* 是否使能同步传输模式下的等待信号,此处未用到 */
g_sram_handle.Init.AsynchronousWait = FSMC_ASYNCHRONOUS_WAIT_DISABLE;
g_sram_handle.Init.WriteBurst = FSMC_WRITE_BURST_DISABLE; /* 禁止突发写 */

/*FSMC 读时序控制寄存器*/
/* 地址建立时间(ADDSET)为 1 个 HCLK 1/72M=13.9ns */
fsmc_read_handle.AddressSetupTime = 0;
fsmc_read_handle.AddressHoldTime = 0;
/* 数据保存时间(DATAST)为 16 个 HCLK 13.9*16=222.4ns */
fsmc_read_handle.DataSetupTime = 15; /* 部分液晶驱动 IC 读数据时,速度不能太快 */
fsmc_read_handle.AccessMode = FSMC_ACCESS_MODE_A; /* 模式 A */

/*FSMC 写时序控制寄存器*/
/* 地址建立时间(ADDSET)为 1 个 HCLK 13.9ns */
fsmc_write_handle.AddressSetupTime = 0;
fsmc_write_handle.AddressHoldTime = 0;
/* 数据保存时间(DATAST)为 2 个 HCLK 13.9*2= 27.8ns */
fsmc_write_handle.DataSetupTime = 1;
fsmc_write_handle.AccessMode = FSMC_ACCESS_MODE_A; /* 模式 A */
HAL_SRAM_Init(&g_sram_handle, &fsmc_read_handle, &fsmc_write_handle);
delay_ms(50);

/* 尝试 9341 ID 的读取 */
lcd_wr_regno(0XD3);
lcddev.id = lcd_rd_data(); /* dummy read */
lcddev.id = lcd_rd_data(); /* 读到 0x00 */
lcddev.id = lcd_rd_data(); /* 读取 93 */
lcddev.id <= 8;
lcddev.id |= lcd_rd_data(); /* 读取 41 */

```

```

if (lcddev.id != 0X9341)          /* 不是 9341 , 尝试看看是不是 ST7789 */
{
    lcd_wr_regno(0X04);
    lcddev.id = lcd_rd_data();    /* dummy read */
    lcddev.id = lcd_rd_data();    /* 读到 0X85 */
    lcddev.id = lcd_rd_data();    /* 读取 0X85 */
    lcddev.id <= 8;
    lcddev.id |= lcd_rd_data();    /* 读取 0X52 */

    if (lcddev.id == 0X8552)      /* 将 8552 的 ID 转换成 7789 */
    {
        lcddev.id = 0x7789;
    }

    if (lcddev.id != 0x7789)      /* 也不是 ST7789, 尝试是不是 NT35310 */
    {
        lcd_wr_regno(0XD4);
        lcddev.id = lcd_rd_data(); /* dummy read */
        lcddev.id = lcd_rd_data(); /* 读回 0X01 */
        lcddev.id = lcd_rd_data(); /* 读回 0X53 */
        lcddev.id <= 8;
        lcddev.id |= lcd_rd_data(); /* 这里读回 0X10 */

        if (lcddev.id != 0X5310)  /* 也不是 NT35310, 尝试看看是不是 NT35510 */
        {
            /* 发送密钥 (厂家提供, 照搬即可) */
            lcd_write_reg(0xF000, 0x0055);
            lcd_write_reg(0xF001, 0x00AA);
            lcd_write_reg(0xF002, 0x0052);
            lcd_write_reg(0xF003, 0x0008);
            lcd_write_reg(0xF004, 0x0001);

            lcd_wr_regno(0xC500);    /* 读取 ID 高 8 位 */
            lcddev.id = lcd_rd_data(); /* 读回 0X55 */
            lcddev.id <= 8;

            lcd_wr_regno(0xC501);    /* 读取 ID 低 8 位 */
            lcddev.id |= lcd_rd_data(); /* 读回 0X10 */
            delay_ms(5);

            if (lcddev.id != 0X5510)  /* 也不是 NT5510, 尝试看看是不是 SSD1963 */
            {
                lcd_wr_regno(0XA1);
                lcddev.id = lcd_rd_data();
                lcddev.id = lcd_rd_data(); /* 读回 0X57 */
                lcddev.id <= 8;
                lcddev.id |= lcd_rd_data(); /* 读回 0X61 */
                /* SSD1963 读回的 ID 是 5761H, 为方便区分, 我们强制设置为 1963 */
                if (lcddev.id == 0X5761) lcddev.id = 0X1963;
            }
        }
    }
}

/* 特别注意, 如果在 main 函数里面屏蔽串口 1 初始化, 则会卡死在 printf
 * 里面 (卡死在 fputc 函数), 所以, 必须初始化串口 1, 或者屏蔽掉下面
 * 这行 printf 语句 !!!!!!!
 */
printf("LCD ID:%x\r\n", lcddev.id); /* 打印 LCD ID */

if (lcddev.id == 0X7789)
{

```



```

    lcd_ex_st7789_reginit(); /* 执行 ST7789 初始化 */
}
else if (lcddev.id == 0X9341)
{
    lcd_ex_ili9341_reginit(); /* 执行 ILI9341 初始化 */
}
else if (lcddev.id == 0x5310)
{
    lcd_ex_nt35310_reginit(); /* 执行 NT35310 初始化 */
}
else if (lcddev.id == 0x5510)
{
    lcd_ex_nt35510_reginit(); /* 执行 NT35510 初始化 */
}
else if (lcddev.id == 0X1963)
{
    lcd_ex_ssd1963_reginit(); /* 执行 SSD1963 初始化 */
    lcd_ssd_backlight_set(100); /* 背光设置为最亮 */
}

lcd_display_dir(0); /* 默认为竖屏 */
LCD_BL(1); /* 点亮背光 */
lcd_clear(WHITE);
}

```

该函数先对 FSMC 相关 IO 进行初始化，然后使用 HAL_SRAM_Init 函数初始化 FSMC 控制器，同时我们使用 HAL_SRAM_MspInit 回调函数来初始化相应的 IO 口，最后读取 LCD 控制器的型号，根据控制 IC 的型号执行不同的初始化代码，这样提高了整个程序的通用性。为了简化 lcd.c 的初始化程序，不同控制 IC 的芯片对应的初始化程序（如：lcd_ex_st7789_reginit()、lcd_ex_ili9341_reginit()等）我们放在 lcd_ex.c 文件中，这些初始化代码完成对 LCD 寄存器的初始化，由 LCD 厂家提供，一般是不需要做任何修改的，我们直接调用就可以了。

下面是 6 个简单，但是很重要的函数：

```

/**
 * @brief      LCD 写数据
 * @param      data: 要写入的数据
 * @retval     无
 */
void lcd_wr_data(volatile uint16_t data)
{
    data = data; /* 使用-O2 优化的时候,必须插入的延时 */
    LCD->LCD_RAM = data;
}

/**
 * @brief      LCD 写寄存器编号/地址函数
 * @param      regno: 寄存器编号/地址
 * @retval     无
 */
void lcd_wr_regno(volatile uint16_t regno)
{
    regno = regno; /* 使用-O2 优化的时候,必须插入的延时 */
    LCD->LCD_REG = regno; /* 写入要写的寄存器序号 */
}

/**
 * @brief      LCD 写寄存器
 * @param      regno: 寄存器编号/地址
 * @param      data: 要写入的数据
 * @retval     无
 */
void lcd_write_reg(uint16_t regno, uint16_t data)
{
}

```

```

LCD->LCD_REG = regno; /* 写入要写的寄存器序号 */
LCD->LCD_RAM = data; /* 写入数据 */
}

/**
 * @brief      LCD 延时函数,仅用于部分在 mdk -O1 时间优化时需要设置的地方
 * @param      t:延时的数值
 * @retval     无
 */
static void lcd_opt_delay(uint32_t i)
{
    while (i--); /*使用 AC6 时空循环可能被优化,可使用 while(1) __asm volatile(""); */
}

/**
 * @brief      LCD 读数据
 * @param      无
 * @retval     读取到的数据
 */
static uint16_t lcd_rd_data(void)
{
    volatile uint16_t ram; /* 防止被优化 */
    lcd_opt_delay(2);
    ram = LCD->LCD_RAM;
    return ram;
}

/**
 * @brief      准备写 GRAM
 * @param      无
 * @retval     无
 */
void lcd_write_ram_prepare(void)
{
    LCD->LCD_REG = lcddev.wramcmd;
}

```

因为 FSMC 自动控制了 WR/RD/CS 等这些信号,所以这 6 个函数实现起来都非常简单,我们就不多说,注意,上面有几个函数,我们添加了一些对 MDK -O2 优化的支持,去掉的话,在-O2 优化的时候会出问题。这些函数实现功能见函数前面的备注,通过这几个简单函数的组合,我们就可以对 LCD 进行各种操作了。

下面要介绍的函数是坐标设置函数,该函数代码如下:

```

/**
 * @brief      设置光标位置(对 RGB 屏无效)
 * @param      x,y: 坐标
 * @retval     无
 */
void lcd_set_cursor(uint16_t x, uint16_t y)
{
    if (lcddev.id == 0X1963)
    {
        if (lcddev.dir == 0) /* 竖屏模式, x 坐标需要变换 */
        {
            x = lcddev.width - 1 - x;
            lcd_wr_regno(lcddev.setxcmd);
            lcd_wr_data(0);
            lcd_wr_data(0);
            lcd_wr_data(x >> 8);
            lcd_wr_data(x & 0XFF);
        }
        else /* 横屏模式 */
        {
            lcd_wr_regno(lcddev.setxcmd);

```

```

        lcd_wr_data(x >> 8);
        lcd_wr_data(x & 0xFF);
        lcd_wr_data((lcddev.width - 1) >> 8);
        lcd_wr_data((lcddev.width - 1) & 0xFF);
    }

    lcd_wr_regno(lcddev.setycmd);
    lcd_wr_data(y >> 8);
    lcd_wr_data(y & 0xFF);
    lcd_wr_data((lcddev.height - 1) >> 8);
    lcd_wr_data((lcddev.height - 1) & 0xFF);
}

else if (lcddev.id == 0X5510)
{
    lcd_wr_regno(lcddev.setxcmd);
    lcd_wr_data(x >> 8);
    lcd_wr_regno(lcddev.setxcmd + 1);
    lcd_wr_data(x & 0xFF);
    lcd_wr_regno(lcddev.setycmd);
    lcd_wr_data(y >> 8);
    lcd_wr_regno(lcddev.setycmd + 1);
    lcd_wr_data(y & 0xFF);
}

else /* 9341/5310/7789 等 设置坐标 */
{
    lcd_wr_regno(lcddev.setxcmd);
    lcd_wr_data(x >> 8);
    lcd_wr_data(x & 0xFF);
    lcd_wr_regno(lcddev.setycmd);
    lcd_wr_data(y >> 8);
    lcd_wr_data(y & 0xFF);
}
}

```

该函数实现将 LCD 的当前操作点设置到指定坐标(x,y)。因为 9341/5310/1963/5510 等的设置有些不太一样，所以进行了区别对待。

接下来介绍画点函数，其定义如下：

```

/**
 * @brief      画点
 * @param      x,y: 坐标
 * @param      color: 点的颜色(32 位颜色,方便兼容 LTDC)
 * @retval     无
 */
void lcd_draw_point(uint16_t x, uint16_t y, uint32_t color)
{
    lcd_set_cursor(x, y); /* 设置光标位置 */
    lcd_write_ram_prepare(); /* 开始写入 GRAM */
    LCD->LCD_RAM = color;
}

```

该函数实现比较简单，就是先设置坐标，然后往坐标写颜色。lcd_draw_point 函数虽然简单，但是至关重要，其他几乎所有上层函数，都是通过调用这个函数实现的。

下面介绍读点函数，用于读取 LCD 的 GRAM，这里说明一下，为什么 OLED 模块没做读 GRAM 的函数，而这里做了。因为 OLED 模块是单色的，所需要全部 GRAM 也就 1K 个字节，而 TFTLCD 模块为彩色的，点数也比 OLED 模块多很多，以 16 位色计算，一款 320×240 的液晶，需要 320×240×2 个字节来存储颜色值，也就是也需要 150K 字节，这对任何一款单片机来说，都不是一个小数目了。而且我们在图形叠加的时候，可以先读回原来的值，然后写入新的值，在完成叠加后，我们又恢复原来的值。这样在做一些简单菜单的时候，是很有用的。这里我们读取 TFTLCD 模块数据的函数为 LCD_ReadPoint，该函数直接返回读到的 GRAM 值。该函数使用之前要先设置读取的 GRAM 地址，通过 lcd_set_cursor 函数来实现。lcd_read_point 的代码如下：

```

/**

```

```

* @brief      读取个某点的颜色值
* @param      x,y:坐标
* @retval     此点的颜色(32 位颜色,方便兼容 LTDC)
*/
uint32_t lcd_read_point(uint16_t x, uint16_t y)
{
    uint16_t r = 0, g = 0, b = 0;

    if (x >= lcddev.width || y >= lcddev.height) return 0; /* 超过了范围,直接返回 */

    lcd_set_cursor(x, y); /* 设置坐标 */

    if (lcddev.id == 0X5510)
    {
        lcd_wr_regno(0X2E00); /* 5510 发送读 GRAM 指令 */
    }
    else
    {
        lcd_wr_regno(0X2E); /* 9341/5310/1963/7789 等发送读 GRAM 指令 */
    }

    r = lcd_rd_data(); /* 假读(dummy read) */

    if (lcddev.id == 0X1963) return r; /* 1963 直接读就可以 */

    r = lcd_rd_data(); /* 实际坐标颜色 */

    /* 9341/5310/5510/7789 要分 2 次读出 */
    b = lcd_rd_data();
    /* 对于 9341/5310/5510/7789, 第一次读取的是 RG 的值,R 在前,G 在后,各占 8 位 */
    g = r & 0XFF;
    g <<= 8;
    /* 9341/5310/5510/7789 需要公式转换一下 */
    return (((r >> 11) << 11) | ((g >> 10) << 5) | (b >> 11));
}

```

在 lcd_read_point 函数中,因为我们的代码不止支持一种 LCD 驱动器,所以,我们根据不同的 LCD 驱动器(lcddev.id)型号,执行不同的操作,以实现对各个驱动器兼容,提高函数的通用性。

第十个要介绍的是字符显示函数 lcd_show_char,该函数同前面 OLED 模块的字符显示函数差不多,但是这里的字符显示函数多了 1 个功能,就是可以以叠加方式显示,或者以非叠加方式显示。叠加方式显示多用于在显示的图片上再显示字符。非叠加方式一般用于普通的显示。该函数实现代码如下:

```

/**
* @brief      在指定位置显示一个字符
* @param      x,y : 坐标
* @param      chr : 要显示的字符:" "---->"~"
* @param      size : 字体大小 12/16/24/32
* @param      mode : 叠加方式(1);非叠加方式(0);
* @retval     无
*/
void lcd_show_char(uint16_t x, uint16_t y, char chr, uint8_t size,
                  uint8_t mode, uint16_t color)
{
    uint8_t temp, t1, t;
    uint16_t y0 = y;
    uint8_t csize = 0;
    uint8_t *pfont = 0;
    /* 得到字体一个字符对应点阵集所占的字节数 */
    csize = (size / 8 + ((size % 8) ? 1 : 0)) * (size / 2);
    /* 得到偏移后的值(ASCII 字库是从空格开始取模,所以-' '就是对应字符的字库) */
    chr = chr - ' ';
}

```

```

switch (size)
{
    case 12:
        pfont = (uint8_t *)asc2_1206[chr]; /* 调用 1206 字体 */
        break;

    case 16:
        pfont = (uint8_t *)asc2_1608[chr]; /* 调用 1608 字体 */
        break;

    case 24:
        pfont = (uint8_t *)asc2_2412[chr]; /* 调用 2412 字体 */
        break;

    case 32:
        pfont = (uint8_t *)asc2_3216[chr]; /* 调用 3216 字体 */
        break;

    default:
        return ;
}

for (t = 0; t < csize; t++)
{
    temp = pfont[t]; /* 获取字符的点阵数据 */

    for (t1 = 0; t1 < 8; t1++) /* 一个字节 8 个点 */
    {
        if (temp & 0x80) /* 有效点,需要显示 */
        {
            lcd_draw_point(x, y, color); /* 画点出来,要显示这个点 */
        }
        else if (mode == 0) /* 无效点,不显示 */
        {
            /* 画背景色,相当于这个点不显示(注意背景色由全局变量控制) */
            lcd_draw_point(x, y, g_back_color);
        }

        temp <<= 1; /* 移位,以便获取下一个位的状态 */
        y++;

        if (y >= lcddev.height)return; /* 超区域了 */

        if ((y - y0) == size) /* 显示完一行了? */
        {
            y = y0; /* y 坐标复位 */
            x++; /* x 坐标递增 */

            if (x >= lcddev.width)return; /* x 坐标超区域了 */

            break;
        }
    }
}
}

```

在 lcd_show_char 函数里面,我们用到了四个字符集点阵数据数组 asc2_1206、asc2_1608、asc2_2412 和 asc2_3216。

lcd.c 的函数比较多,其他的函数请大家自行查看源码,都有详细的注释。

2. main.c 代码

在 main.c 里面编写如下代码:


```
int main(void)
{
    uint8_t x = 0;
    uint8_t lcd_id[12];

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(RCC_PLL_MUL9); /* 设置时钟, 72Mhz */
    delay_init(72); /* 延时初始化 */
    usart_init(115200); /* 串口初始化为 115200 */
    led_init(); /* 初始化 LED */
    lcd_init(); /* 初始化 LCD */

    g_point_color = RED;
    sprintf((char *)lcd_id, "LCD ID:%04X", lcddev.id); /* 将 id 打印到 lcd_id 数组 */

    while (1)
    {
        switch (x)
        {
            case 0: lcd_clear(WHITE); break;
            case 1: lcd_clear(BLACK); break;
            case 2: lcd_clear(BLUE); break;
            case 3: lcd_clear(RED); break;
            case 4: lcd_clear(MAGENTA); break;
            case 5: lcd_clear(GREEN); break;
            case 6: lcd_clear(CYAN); break;
            case 7: lcd_clear(YELLOW); break;
            case 8: lcd_clear(BRRED); break;
            case 9: lcd_clear(GRAY); break;
            case 10: lcd_clear(LGRAY); break;
            case 11: lcd_clear(BROWN); break;
        }

        lcd_show_string(10, 40, 240, 32, 32, "STM32", RED);
        lcd_show_string(10, 80, 240, 24, 24, "TFTLCD TEST", RED);
        lcd_show_string(10, 110, 240, 16, 16, "ATOM@ALIENTEK", RED);
        lcd_show_string(10, 130, 240, 16, 16, (char *)lcd_id, RED); /*显示 LCD_ID*/

        x++;
        if (x == 12)
            x = 0;

        LED0_TOGGLE(); /* 红灯闪烁 */
        delay_ms(1000);
    }
}
```

main 函数功能主要是显示一些固定的字符，字体大小包括 32*16、24*12、16*8 和 12*6 四种，同时显示 LCD 驱动 IC 的型号，然后不停的切换背景颜色，每 1s 切换一次。而 LED0 也会不停的闪烁，指示程序已经在运行了。其中我们用到一个 sprintf 的函数，该函数用法同 printf，只是 sprintf 把打印内容输出到指定的内存区间上，最终在死循环中通过 lcd_show_string 函数进行屏幕显示，sprintf 的详细用法，请百度学习。

特别注意：

usart_init 函数，不能去掉，因为在 lcd_init 函数里面调用了 printf，所以一旦去掉这个初始化，就会死机！实际上，只要你的代码有用到 printf，就必须初始化串口，否则都会死机，停在 usart.c 里面的 fputc 函数出不来。

25.4 下载验证

下载代码后，LED0 不停的闪烁，提示程序已经在运行了。同时可以看到 TFTLCD 模块的显示背景色不停切换，如图 25.4.1 所示：

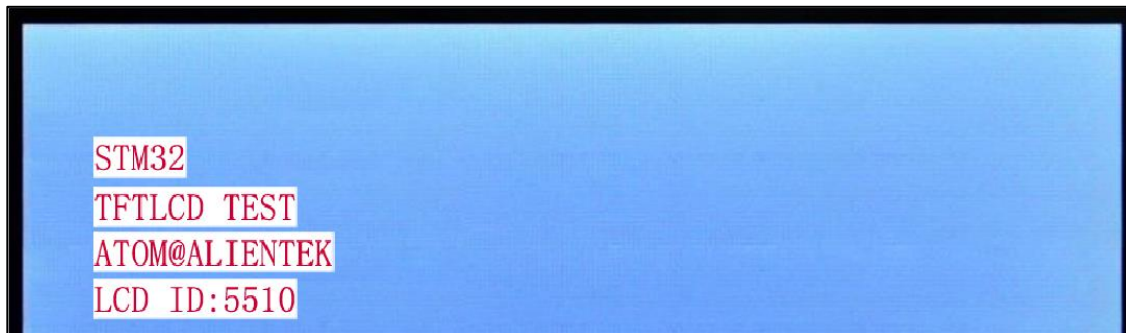


图 25.4.1 TFTLCD 显示效果图

此外，为了让大家能直观的了解 LCD 屏的扫描方式，我们额外编写了两个 main.c 文件（main1.c 和 main2.c，放到 User 文件夹中），方便大家编译下载，观察现象。

使用方法：关闭工程后，先把原实验中的 main.c 改成其他名字，然后把 main1.c 重命名为 main.c，双击 keilkill.bat 清理编译的中间文件，最后打开工程重新编译下载，就可以观察实验现象。观察了 main1.c，可以再观察 main2.c，main2.c 文件的操作方法类似。这两个 main.c 文件的程序非常简单，这里就不讲解，具体请看源码。